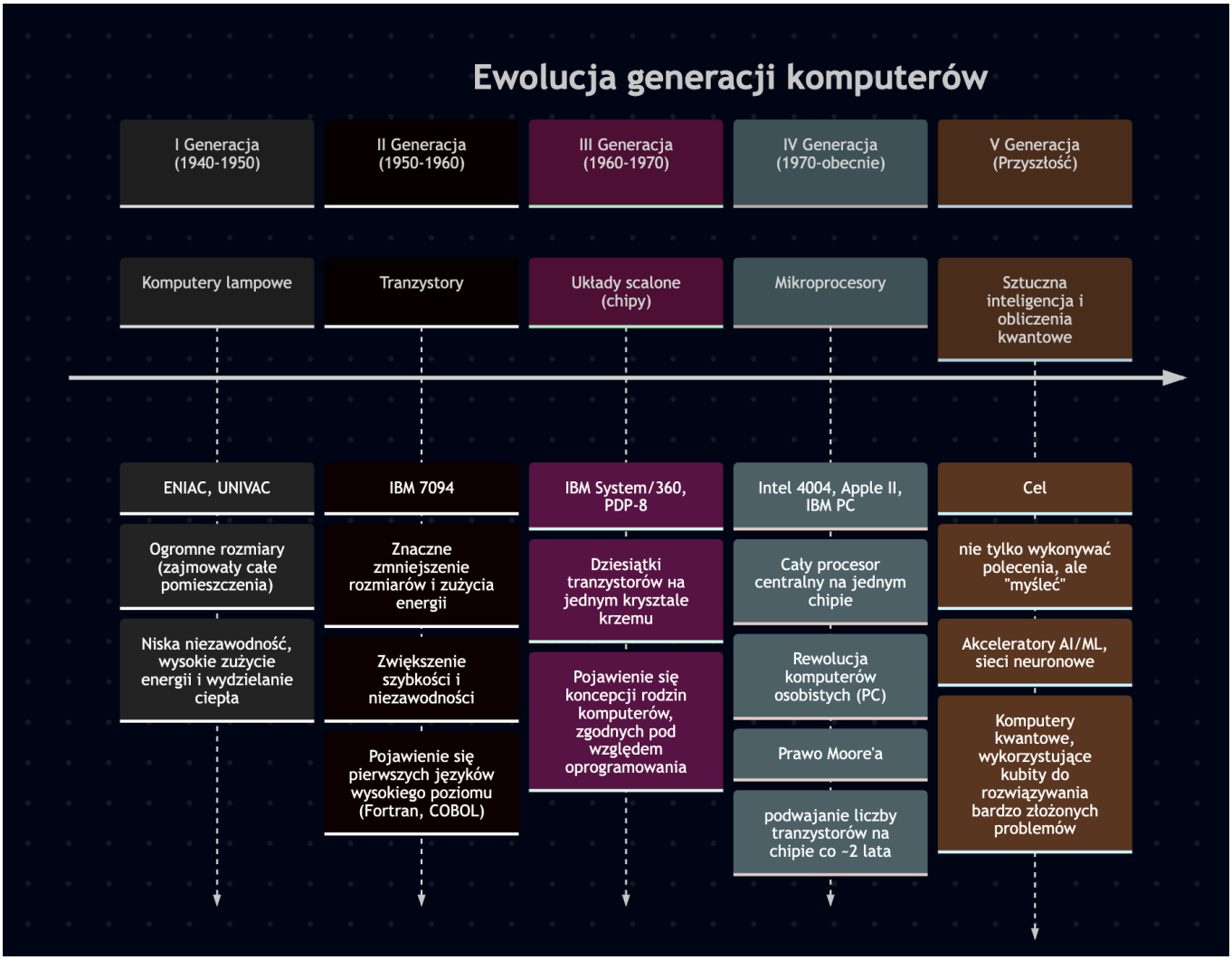
**Lekcja 1: Podstawy algorytmizacji i programowania

`#lekcja` `#python` `#podstawy` `#terminal` `#vscode`

1. Rozwój architektury komputerowej

Zanim nauczymy się rozmawiać z komputerem, warto dowiedzieć się, jaką drogę przebył od gigantycznych mechanicznych potworów do urządzeń mieszczących się w naszej kieszeni. Ta droga dzieli się na generacje, z których każda była definiowana przez kluczową technologię.

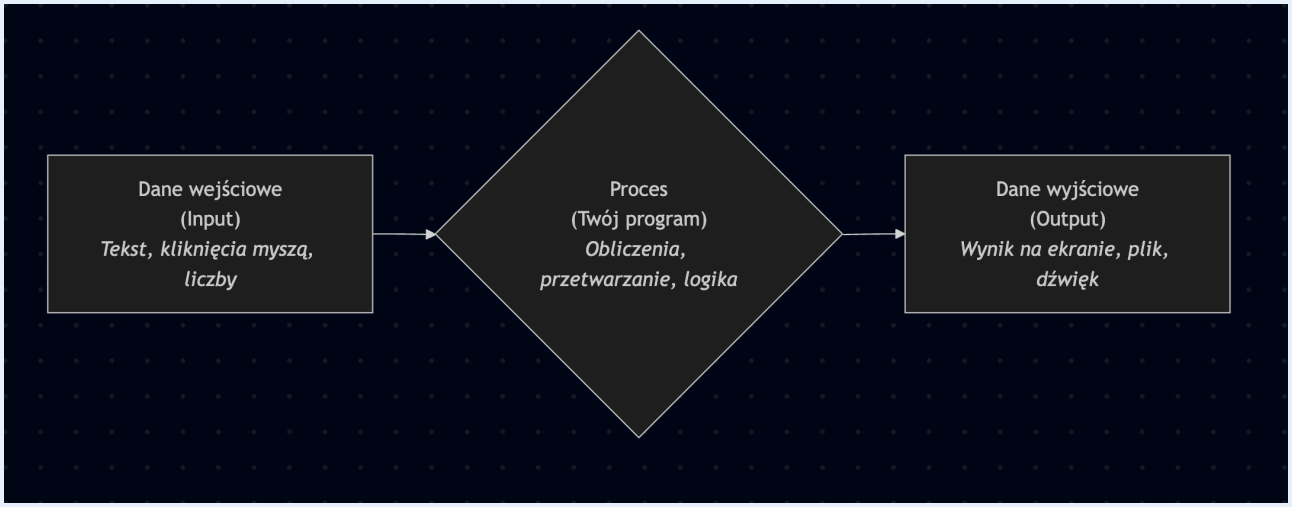


> **Kluczowa idea**
>
> Każda nowa generacja sprawiała, że komputery stawały się mniejsze, szybsze, tańsze i bardziej dostępne, co ostatecznie doprowadziło do ery cyfrowej, w której żyjemy.

2. O programowaniu w ogóle

Definition

Programowanie — to proces tworzenia precyzyjnego i jednoznacznego zestawu instrukcji (programu) dla komputera, aby mógł on wykonać określone zadanie.



Quote

Programista to tłumacz z języka ludzkiego (pomysły, zadania) na język zrozumiały dla komputera (kod).

3. Powszechne języki programowania

Istnieje ogromna liczba języków programowania. Można je klasyfikować według różnych kryteriów.

Typ	Proces	Analogia	Zalety	Wady	Przykłady
Kompilowane	Cały kod jest tłumaczony na kod maszynowy **z góry** i zapisywany w pliku wykonywalnym (`.exe`).	Tłumacz przekłada całą książkę naraz, tworząc gotową książkę w innym języku.	Wysoka szybkość wykonania.	Potrzebny jest osobny krok kompilacji. Trudniej przenosić między systemami operacyjnymi.	`C++` , `Java` , `Go` , `Rust`
Interpretowane	Kod jest wykonywany **linijka po**	Tłumacz symultaniczny przekłada	Łatwość wprowadzania zmian i	Działają wolniej, ponieważ	`Python` , `JavaScript` , `Ruby`

	linijce przez specjalny program — interpreter.	Twoją mowę zdanie po zdaniu w czasie rzeczywistym.	testowania. Łatwiejsze przenoszenie ().	tłumaczenie odbywa się w trakcie wykonania.	

Dlaczego Python to świetny wybór na start?

1. **Prosta i czytelna składnia:** Kod w Pythonie często wygląda jak pseudokod.
2. **Język interpretowany:** Łatwo i szybko można uruchomić kod i zobaczyć wynik.
3. **Ogromna społeczność:** Łatwo znaleźć odpowiedzi na wszelkie pytania.
4. **Wszechstronność:** Nadaje się do tworzenia stron internetowych, analizy danych, uczenia maszynowego, automatyzacji i wielu innych.

4. Instalacja Pythona

Zanim zaczniemy pisać kod, musimy zainstalować sam język Python na naszym komputerze. Jest to program-interpreter, który będzie czytał i wykonywał nasze skrypty.

Windows

1. **Pobierz instalator:** Przejdź na oficjalną stronę python.org/downloads/. Strona automatycznie zaproponuje Ci najnowszą wersję dla systemu Windows.
2. **Uruchom instalację:** Otwórz pobrany plik `.exe`.
3. **WAŻNY KROK:** W pierwszym oknie instalatora **koniecznie** zaznacz pole na dole z napisem **"Add Python to PATH"** lub **"Add python.exe to PATH"**.

   Dlaczego to takie ważne?
   Ta opcja pozwala na uruchamianie Pythona z dowolnego miejsca w terminalu, po prostu wpisując polecenie python. Bez niej musiałbyś za każdym razem podawać pełną ścieżkę do pliku python.exe, co jest bardzo niewygodne.

4. Kliknij **"Install Now"** i postępuj zgodnie z instrukcjami.

macOS

Na macOS jest już zainstalowana systemowa wersja Pythona, ale często jest ona przestarzała. Lepiej zainstalować świeżą wersję osobno.

1. **Zainstaluj Homebrew:** Jeśli nie masz jeszcze menedżera pakietów Homebrew, zainstaluj go. To standard dla programistów na Macu. Otwórz terminal i wykonaj polecenie z oficjalnej strony brew.sh.
2. **Zainstaluj Pythona:** Otwórz terminal i wykonaj proste polecenie:

   ```
   brew install python
   ```

   Homebrew zrobi wszystko za Ciebie.

Linux

Na większości dystrybucji Linuksa Python jest już zainstalowany. Możesz to sprawdzić i w razie potrzeby zaktualizować.

- **Debian/Ubuntu:**

  ```
  sudo apt update
  sudo apt install python3
  ```

- **Fedora/CentOS:**

  ```
  sudo dnf install python3
  ```

Sprawdzenie instalacji (dla wszystkich systemów)

Aby upewnić się, że wszystko poszło pomyślnie, otwórz nowy terminal (lub uruchom ponownie stary) i wpisz polecenie:

```
python --version
```

Lub, na niektórych systemach (macOS, Linux):

```
python3 --version
```

Jeśli zobaczysz numer wersji (np. `Python 3.11.4`), oznacza to, że wszystko zostało zainstalowane poprawnie. Gratulacje!

5. Podstawy pracy z terminalem (wierszem poleceń)

Terminal (lub wiersz poleceń) to jedno z najważniejszych narzędzi programisty. To sposób na zarządzanie komputerem za pomocą poleceń tekstowych, a nie kliknięć myszą.

Jak otworzyć terminal?

- **Windows:** `PowerShell` (zalecane). Naciśnij `Win + X` i wybierz "Windows PowerShell" lub "Terminal".
- **macOS:** Aplikacja `Terminal.app`. Znajduje się w `Programy > Narzędzia` lub przez wyszukiwarkę Spotlight (`Cmd + Spacja`).
- **Linux:** Zależy od dystrybucji, zwykle `Ctrl + Alt + T`.

Podstawowe polecenia, które warto znać

Polecenie (Windows)	Polecenie (macOS/Linux)	Opis
`dir`	`ls`	**Lis**t - Pokaż zawartość bieżącego folderu.
`echo %cd%`	`pwd`	**P**rint **W**orking **D**irectory - Pokaż, w którym folderze się teraz znajduję.
`cd <nazwa_folderu>`	`cd <nazwa_folderu>`	**C**hange **D**irectory - Przejdź do wskazanego folderu.
`cd ..`	`cd ..`	Przejdź o jeden poziom wyżej (do folderu nadrzędnego).
`mkdir <nazwa_folderu>`	`mkdir <nazwa_folderu>`	**Ma**ke **Dir**ectory - Utwórz nowy folder.
`cls`	`clear`	Wyczyść ekran terminala.
`python`	`python3`	Uruchom tryb interaktywny Pythona.

6. Instalacja i konfiguracja VSCode

Visual Studio Code (VSCode) — to darmowy, potężny i bardzo popularny edytor kodu od firmy Microsoft.

Krok 1: Pobieranie i instalacja

1. Przejdź na oficjalną stronę: code.visualstudio.com.
2. Strona automatycznie wykryje Twój system operacyjny i zaproponuje odpowiedni plik. Pobierz go.
3. Uruchom instalator. **Ważne:** w systemie Windows upewnij się, że wszystkie pola wyboru, zwłaszcza **"Add to PATH"**, są zaznaczone. Pozwoli to na uruchamianie VSCode z terminala za pomocą polecenia `code .`.

Krok 2: Instalacja kluczowych rozszerzeń

1. Otwórz VSCode.

2. Po lewej stronie znajdź ikonę z klockami (Extensions) lub naciśnij `Ctrl+Shift+X`.
3. W polu wyszukiwania znajdź i zainstaluj:
   - **Python** (od Microsoft): najważniejsze rozszerzenie. Dodaje podświetlanie składni, autouzupełnianie, debugowanie itp.
   - **Pylance** (od Microsoft): zazwyczaj instaluje się razem z Pythonem. Sprawia, że analiza kodu jest jeszcze inteligentniejsza.
   - **(Opcjonalnie) Material Icon Theme**: upiększa ikony plików i folderów.

Krok 3: Konfiguracja przestrzeni roboczej

1. **Otwórz folder projektu:** Utwórz folder na lekcje (`python_lessons`) i otwórz go w VSCode (`File > Open Folder`).
2. **Otwórz wbudowany terminal:** Naciśnij `Ctrl+` ` (odwrotny apostrof, klawisz z tyldą).
3. **Wybierz interpreter Pythona:**
   - Naciśnij `Ctrl+Shift+P`.
   - Wpisz `Python: Select Interpreter`.
   - Wybierz wersję Pythona, którą zainstalowałeś w kroku 4.

7. Algorytmy

> **Algorytm — to skończony ciąg jednoznacznie zdefiniowanych kroków, niezbędnych do rozwiązania zadania.**

Przykład: Algorytm "Przejście przez ulicę na światłach"

```mermaid graph TD A[Podejdź do przejścia dla pieszych] --> B{Czy świeci się zielone światło?}; B -- Tak --> C[Przejdź przez ulicę]; B -- Nie --> D{Czy świeci się czerwone światło?}; D -- Tak --> E[Czekaj]; E --> B; D -- Nie (miga zielone) --> F[Szybko oceń sytuację]; F --> B; C --> G[Koniec]; ```

8. Systemy liczbowe

Ludzie używają systemu dziesiętnego (0-9). Komputery — systemu dwójkowego (0, 1).

Konwersja w Pythonie

```
# Konwersja z systemu dziesiętnego na inne
liczba_dziesietna = 255
print(bin(liczba_dziesietna))  # '0b11111111'
print(hex(liczba_dziesietna))  # '0xff'
print(oct(liczba_dziesietna))  # '0o377'
```
```

```
# Konwersja z innych systemów na dziesiętny
lancuch_binarny = "1010"
wynik_dziesietny = int(lancuch_binarny, 2)
print(wynik_dziesietny) # 10
```

9. Reprezentacja danych w komputerze

Komputer przechowuje **wszystko** jako ciągi zer i jedynek.

- **Tekst:** `A` → kod ASCII `65` → w systemie dwójkowym `01000001`.
- **Obrazy:** Obraz → piksele → kolor piksela (RGB) → liczby w kodzie dwójkowym.
- **Dźwięk:** Fala dźwiękowa → próbki (liczby) → liczby w kodzie dwójkowym.

10. Podstawy algebry logiki (algebry Boole'a)

To matematyka dwóch wartości: **Prawda (`True`)** i **Fałsz (`False`)**.

Operator	Nazwa w Pythonie	Nazwa w logice	Opis
I	`and`	Koniunkcja (	A∧B
LUB	`or`	Alternatywa (	A∨B
NIE	`not`	Negacja (	¬A

Przykłady w Pythonie

```
wiek = 20
ma_prawo_jazdy = True
jest_weekend = True
ma_pieniadze = False

# Czy można prowadzić samochód? Wiek 18+ ORAZ posiadanie prawa jazdy.
moze_prowadzic = wiek >= 18 and ma_prawo_jazdy
print(f"Może prowadzić samochód: {moze_prowadzic}")

# Idziemy do kina? Jeśli jest weekend LUB są pieniądze.
idziemy_do_kina = jest_weekend or ma_pieniadze
print(f"Idziemy do kina: {idziemy_do_kina}")
```

🧪 11. Zadania praktyczne (10 zadań)

Poziom 1: Konfiguracja i pierwsze kroki

1. **Instalacja i "Hello, World!":**
   - Postępując zgodnie z instrukcjami z lekcji, zainstaluj Pythona i VSCode.
   - Utwórz folder `python_lessons` i otwórz go w VSCode.
   - Utwórz plik `task1_hello.py`.
   - Napisz w nim kod `print("Witaj w Pythonie!")` i uruchom go.
2. **Zapoznanie:**
   - Utwórz plik `task2_greeting.py`.
   - Napisz program, który zapyta użytkownika o jego imię i wiek.
   - Wyświetl powitanie, na przykład: `"Cześć, Maria! Wiem, że masz 25 lat."`
3. **Praca z terminalem:**
   - W terminalu VSCode utwórz folder `homework` (`mkdir homework`).
   - Przejdź do niego (`cd homework`).
   - Upewnij się, że jesteś w odpowiednim folderze (`pwd` lub `echo %cd%`).
   - Wróć z powrotem (`cd ..`).

Poziom 2: Praca z danymi i arytmetyka

4. **Kalkulator pola powierzchni:**
   - Utwórz plik `task4_area.py`.
   - Program powinien poprosić o długość i szerokość prostokąta.
   - Oblicz i wyświetl jego pole powierzchni.
   - *Nie zapomnij o `int()` lub `float()`.*
5. **Kantor wymiany walut:**
   - Utwórz plik `task5_exchange.py`.
   - Załóż, że kurs dolara wynosi 4.0 zł. Zapytaj użytkownika o kwotę w złotówkach.
   - Oblicz i wyświetl, ile to będzie w dolarach.
6. **Konwerter systemów liczbowych:**
   - Utwórz plik `task6_converter.py`.
   - Poproś użytkownika o liczbę całkowitą i wyświetl ją w formacie dwójkowym i szesnastkowym.
7. **Odwracacz słów:**
   - Utwórz plik `task7_reverse.py`.
   - Poproś użytkownika o słowo i wyświetl je od tyłu.
   - *Wskazówka: wycinek `s[::-1]`.*

Poziom 3: Logika i proste algorytmy

8. **Sprawdzanie parzystości:**
   - Utwórz plik `task8_even_odd.py`.

- Poproś o liczbę całkowitą i sprawdź, czy jest parzysta.
- *Wskazówka: operator* `% 2`*.*

9. **Prosty kalkulator:**
   - Utwórz plik `task9_calculator.py`.
   - Poproś o dwie liczby i znak (`+`, `-`, `*`, `/`).
   - Wykonaj operację i wyświetl wynik.
   - *Użyj* `if/elif/else`*.*
10. **Wstęp do parku rozrywki:**
    - Utwórz plik `task10_access.py`.
    - Zasada: wpuszczane są dzieci o wzroście od 120 cm **I** z osobą dorosłą, **LUB** młodzież o wzroście od 160 cm.
    - Zapytaj o wzrost i czy jest opiekun (`"tak"` / `"nie"`).
    - Używając `and` i `or`, określ, czy można wpuścić gościa, i wyświetl `True` lub `False`.